Tarea #4

Ponderación: 2 pts

Tiempo estimado: 5 horas

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
Laboratorio - Arquitectura de Computadores y Ensambladores 1 - A
Auxiliar Carlos Alfredo Barrientos López

1. Marco Formativo

1.1 Valores

Indique un valor y cómo se aplica en el laboratorio.

Perseverancia	¿Cómo se aplica en tu laboratorio?
Perseverancia	El estudiante demuestra perseverancia al enfrentar los desafíos propios del lenguaje ensamblador ARM-64: depurar errores a nivel de registro, entender el comportamiento de la memoria y construir funciones modulares correctas requiere intentos sostenidos y atención al detalle.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Construye una mini-librería modular en ensamblador ARM-64 puro para manipulación de strings, aplicando correctamente syscalls de Linux, convenciones de llamada ABI ARM64 y técnicas de depuración con GDB para producir código funcional, documentado y verificado.
Competencia Específica	1. Distingue los niveles y tipos de excepciones en ARM-64 analizando el manejo de syscalls y directivas de ensamblador para interactuar correctamente con el sistema operativo y gestionar eventos del procesador. 2. Construye funciones reutilizables en ensamblador ARM-64 aplicando convenciones de llamada, manejo del stack y paso de parámetros para modularizar programas y facilitar su mantenimiento y escalabilidad. 3. Depura programas escritos en ensamblador ARM-64 usando GDB para inspeccionar registros, memoria y flujo de ejecución paso a paso con el fin de identificar y corregir errores lógicos en el código de bajo nivel.

1.3 Objetivo

Construir una mini-librería de cinco funciones para manipulación de strings completamente en ensamblador ARM-64 puro (str_len, str_copy, str_compare, str_reverse, str_to_upper), aplicando convenciones de llamada ABI ARM64, manejo correcto del stack y syscalls de Linux (read/write), e integrarlas en un programa principal interactivo; depurando mínimo tres

funciones con GDB para verificar el estado de registros y memoria, y documentando el proceso en un reporte técnico de una página.

Verbos de acción aplicados: construir, implementar, aplicar, depurar, documentar, verificar.

2. Material de apoyo

• ARM Architecture Reference Manual ARMv8 — A64 Instruction Set: https://developer.arm.com/documentation/ddi0487/latest

• Procedure Call Standard for the ARM 64-bit Architecture (AAPCS64): https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst

• Linux Syscall Table para ARM64: https://arm64.syscall.sh/

• GDB Debugging Guide — inspecting registers and memory: https://sourceware.org/gdb/current/onlinedocs/gdb/

• GNU Assembler (GAS) — directivas y sintaxis ARM: https://sourceware.org/binutils/docs/as/

3. Actividad

El estudiante construirá una mini-librería de funciones para manipulación de strings, implementada completamente en ARM-64 puro. Cada función deberá ser modular, usar el stack correctamente y comunicarse a través de registros siguiendo la ABI ARM64.

Tecnologías y herramientas: Linux (Ubuntu ARM64 o QEMU ARM64), Ensamblador ARM-64 puro (GAS), GDB (depurador), Syscalls Linux (read, write), Make.

Funciones obligatorias a implementar:

1. **str_len** — Calcula la longitud de una cadena terminada en NULL. Entrada: X0 = dirección base del string. Salida: X0 = longitud (sin contar el \0).

2. **str_copy** — Copia una cadena de origen a destino. Entrada: X0 = destino, X1 = origen. Debe manejar el caso de cadena vacía.

3. **str_compare** — Compara dos cadenas carácter a carácter. Entrada: X0 = cadena 1, X1 = cadena 2. Salida: X0 = 0 si son iguales, 1 si difieren.

4. str_reverse — Invierte una cadena in-place. Entrada: X0 = dirección base del string. Debe usar el stack para almacenamiento temporal.

5. str_to_upper — Convierte todos los caracteres a mayúsculas usando operaciones lógicas ARM para transformar cada byte.

Programa principal:

• Leer un string del usuario por consola (syscall read).

• Aplicar y demostrar cada función sobre el string leído.

• Imprimir los resultados de cada operación usando syscall write.

Requisitos técnicos:

• Cada función en su propio bloque con etiqueta y stack frame completo (preservar registros callee-saved).

• Mínimo 3 funciones depuradas con GDB mostrando el estado de registros e inspección de memoria.

• Código completamente comentado línea a línea.

Entregables:

• Código fuente .s debidamente comentado.

• Makefile para compilación y enlazado del proyecto.

• Reporte corto (1 página) explicando cómo funciona cada función de la librería.

• Capturas de GDB mostrando inspección de memoria y registros durante la ejecución de al menos 3 funciones.

Formato de entrega: Repositorio en GitHub/GitLab o archivo comprimido (.zip) subido a la plataforma del curso, con carpetas organizadas (src/, report/, screenshots/).

4. Cronograma

Acción	Número de Semana	Fecha
Asignación de tarea	Semana 8	13/10/2026
Entrega de Tarea	Semana 10	18/10/2026

5. Rúbrica de calificación

5.1. Requisitos para optar a Calificación

Para optar a calificación, el estudiante debe cumplir obligatoriamente con los siguientes requisitos:

1. Entregar las 5 funciones implementadas en ARM-64 puro antes de la fecha límite.

2. El programa debe compilar sin errores usando el Makefile incluido.

3. Presentar capturas de depuración con GDB de al menos 3 funciones mostrando registros y memoria.

4. Incluir el reporte técnico (1 página) explicando el funcionamiento de cada función de la librería.

5.2 Detalle de la Calificación

CRITERIO	DESCRIPCIÓN	PUNTEO
Funciones str_len y str_copy	str_len calcula correctamente la longitud de cualquier string incluyendo cadena vacía. str_copy duplica el contenido al destino manejando el caso vacío. Ambas respetan la ABI ARM64 (parámetros en X0/X1, resultado en X0).	20
Funciones str_compare, str_reverse y str_to_upper	str_compare retorna 0 o 1 correctamente. str_reverse invierte in-place usando el stack. str_to_upper aplica operaciones lógicas ARM para convertir cada byte. Las tres usan stack frames completos y preservan registros callee-saved.	30

Programa principal e integración	El programa principal lee un string por consola (syscall read), llama cada función de forma modular e imprime los resultados (syscall write). El Makefile compila y enlaza sin errores.	20
Depuración con GDB y capturas	Se presentan capturas de sesión GDB para al menos 3 funciones, mostrando claramente el estado de registros (info registers) e inspección de memoria (x/s o x/b) en puntos clave de ejecución.	15
Comentarios y reporte técnico	El código incluye comentarios línea a línea en todas las funciones. El reporte de una página describe claramente el propósito, los registros usados y el flujo de cada función de la librería.	15
TOTAL	Puntaje Total	**100**